



Python 기초 문법 Part2

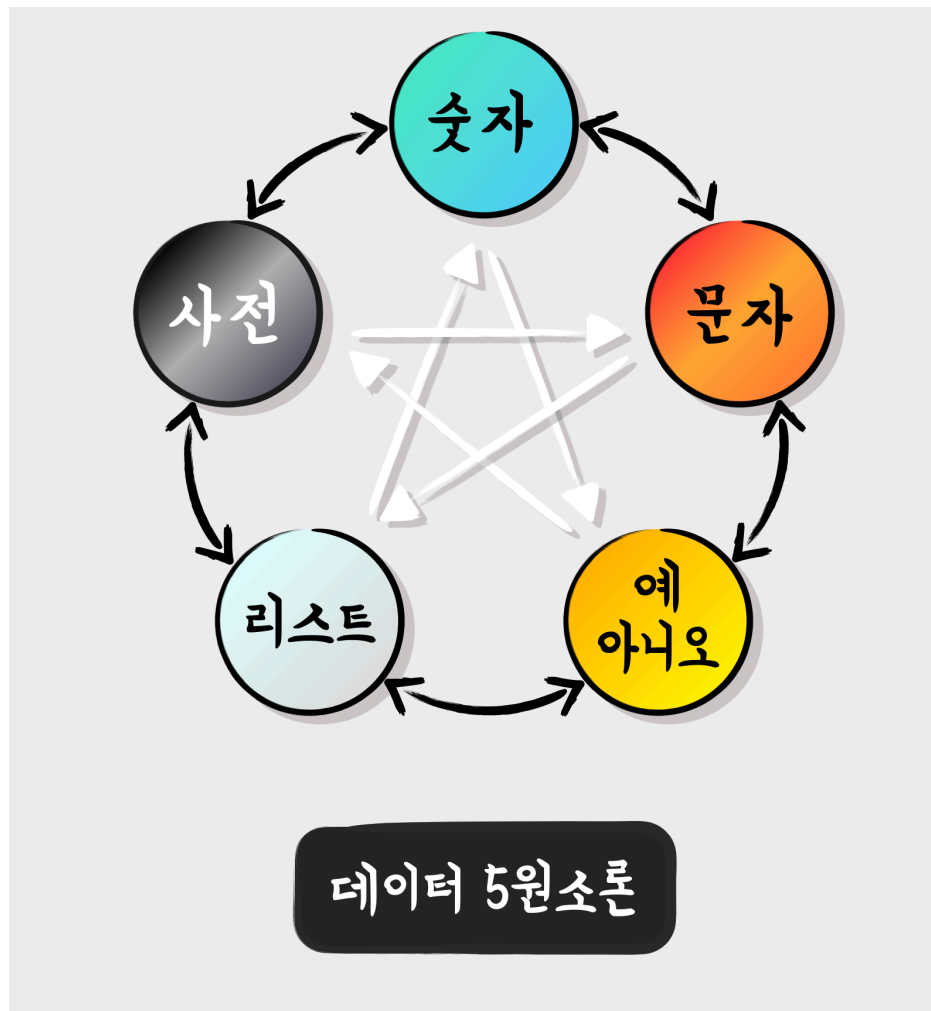


1. 자료 구조 (Data Structure)
2. 리스트
3. 반복문
4. 문자열 다루기

▼ 1. 자료 구조 (Data Structure)



옷장에 옷을 정리하고, 책장에 책을 꽂고, 서랍에 물건을 넣는 것처럼, 자료 구조는 데이터를 체계적으로 정리하고 보관하는 방법입니다.



▼ 자료 구조란

- 단순히 변수만 잘 저장하는 것만으로 자료가 잘 정리될까요?
- 자료 구조는 데이터를 체계적으로 저장하고 관리하는 구조입니다.
- 우리가 물건을 정리할 때 서랍, 상자, 책장 등을 사용하듯이, 프로그래밍에서도 데이터를 담을 특별한 구조가 필요합니다.
- 자료 구조의 종류에 따라 데이터를 찾고, 추가하고, 삭제하는 방식이 달라집니다.

▼ 자료 구조 예시



자료 구조는 멀리 있지 않습니다.

우리가 일상에서 생활하는 데이터들은 모두 특정한 자료구조에 담겨서 구조화 되어 있습니다.

- 인터넷 브라우저 방문 기록: 가장 최근에 방문한 페이지가 맨 위에 쌓이는 구조

- **스마트폰 연락처:** 이름으로 전화번호를 빠르게 찾을 수 있는 구조
- **음악 플레이리스트:** 노래를 장르별로 분류하고 원하는 순서대로 재생할 수 있는 구조

▼ 자료 구조가 필요한 이유



▼ 효율성

책들이 체계적으로 정리된 도서관에서 책을 쉽게 찾을 수 있는 것처럼, 좋은 자료 구조는 **데이터를 빠르게 찾고 관리**할 수 있게 해줍니다.

▼ 확장성

옷장에 새 옷을 추가하기 쉽듯이, 잘 정리된 자료 구조는 새로운 데이터를 쉽게 추가하고 제거할 수 있습니다.

▼ 반대로 얘기하면,

- 적합한 자료 구조를 사용하지 않으면, 데이터를 찾고 관리하는 데 많은 시간과 자원이 낭비됩니다.
- 즉, 프로그램의 성능을 크게 저하시킬 수 있습니다.
- 그래서 코딩 테스트 단골 주제입니다.

▼ 2. 리스트



리스트는 프로그래밍에서 가장 많이 사용되는 자료 구조 중 하나입니다. 마치 설국열차의 객차처럼, 여러 데이터를 순서대로 한 줄로 늘어놓을 수 있습니다. 

▼ 리스트란?

리스트

하루 일과표

1. 사료 먹기
2. 산책 하기
3. 강의 영상 찍기



리스트명 = [값1, 값2, 값3 ...]

```
numbers = [1, 2, 3, 4, 5]
fruits = ["사과", "바나나", "오렌지"]
empty_list = []
mixed = [1, "안녕", True, 3.14] # 다양한 타입 저장 가능
```

- 리스트는 여러 개의 데이터를 순서대로 저장하는 자료구조입니다.
- 예시: 플레이 리스트, 투두 리스트, 등수 등



리스트의 인덱스가 0부터 시작하는 이유?

인덱스의 정확한 의미는 첫 번째 위치로부터의 '거리'를 뜻합니다.

기차를 예로 들어보면,

기관차(시작점)로부터 두 번째 객차까지는 1칸을 이동해야 하고,

세 번째 객차까지는 2칸을 이동해야 합니다.

즉, 인덱스는 '시작점으로부터 얼마나 떨어져 있는지'를 나타내는 거리 개념입니다.



▼ 인덱스(Index) 이해하기

리스트[인덱스]

```
fruits = ["사과", "바나나", "오렌지"]
print(fruits[0])    # 첫 번째 요소: "사과"
print(fruits[1])    # 두 번째 요소: "바나나"
print(fruits[-1])   # 마지막 요소: "오렌지"
```

▼ 특정 위치의 값 찾기

- 인덱스는 리스트에서 각 데이터의 위치를 나타내는 번호입니다.
- 첫 번째 위치는 0번부터 시작합니다.
- 마지막 위치는 -1로도 접근할 수 있습니다.

▼ 특정 값의 위치 찾기

- 리스트에서 특정 요소의 위치를 찾으려면 `index()` 함수를 사용합니다.
- 입력한 값과 일치하는 첫 번째 요소의 위치를 반환합니다.

```
fruits = ["사과", "바나나", "오렌지", "바나나"]
position = fruits.index("바나나") # 1
```

▼ 특정 위치의 값 수정하기

- 리스트의 특정 위치에 있는 값을 다른 값으로 변경할 수 있습니다.
- 기차의 객차를 다른 객차로 교체하는 것처럼, 원하는 위치의 값을 새로운 값으로 바꿀 수 있습니다.

```
fruits = ["사과", "바나나", "오렌지"]
fruits[1] = "딸기"          # ["사과", "딸기", "오렌지"]
fruits = ["사과", "바나나", "오렌지"]
fruits[1] = "딸기"          # ["사과", "딸기", "오렌지"]
```

▼ 특정 범위의 값들 추출하기

리스트에서 특정 범위의 값들을 추출할 때는 `슬라이싱(Slicing)` 을 사용합니다.

```
fruits = [0, 1, 2, 3, 4, 5]

fruits[1:3]    # [1, 2]          (인덱스 1부터 2까지)

fruits[:3]     # [0, 1, 2]       (처음부터 인덱스 2까지)

fruits[2:]     # [2, 3, 4, 5]    (인덱스 2부터 끝까지)
```

▼ 리스트 기본 연산

▼ 1. 리스트 추가하기

리스트에 새로운 요소를 추가하는 방법에는 크게 두 가지가 있습니다.

```
train = ["기관차", "식당칸", "침대칸", "화물칸"]

# append로 맨 뒤에 추가
train.append("꼬리칸")    # ["기관차", "식당칸", "침대칸", "화물칸", "꼬리칸"]

# insert로 원하는 위치에 추가
train.insert(1, "일등칸") # ["기관차", "일등칸", "식당칸", "침대칸", "화물칸", "꼬리칸"]

print(train)
```

▼ append

```
리스트.append(추가할 값)
```

append()를 사용하면 리스트의 맨 뒤에 요소를 추가할 수 있습니다.

▼ insert

```
리스트.insert(추가할 위치, 추가할 값)
```

insert()를 사용하여 원하는 위치에 삽입할 수 있습니다.

▼ 2. 리스트 삭제하기

리스트에서 요소를 삭제하는 방법에는 크게 세 가지가 있습니다.

```
train = ["아이언맨", "토르", "캡틴아메리카", "블랙위도우",
         "호크아이"]

# pop으로 마지막 승객 하차
last_passenger = train.pop()      # "호크아이" 하차

# del로 두 번째 승객 하차
del train[1]                      # "토르" 하차

# remove로 특정 승객 하차
train.remove("캡틴아메리카")     # "캡틴아메리카" 하차
```

▼ pop

리스트.pop()

- pop()을 사용하면 리스트의 마지막 요소를 제거하고 반환합니다.

▼ pop(인덱스)를 사용하면 특정 위치의 요소를 제거하고 반환할 수도 있습니다.

```
fruits = ["사과", "바나나", "오렌지"]
second = fruits.pop(1)          # "바나나" 제거
                                  및 반환
print(fruits)                   # ["사과", "오렌지"]
```

▼ del

del 리스트[인덱스]

del 키워드를 사용하여 특정 위치의 요소를 삭제할 수 있습니다.

▼ remove

```
리스트.remove(삭제할 값)
```

remove()를 사용하여 특정 값을 찾아 삭제할 수 있습니다.

만약 해당 값이 여러 군데에서 발견되면 모두 삭제해줍니다.

▼ 번외: clear

```
리스트.clear()
```

clear()를 사용하면 리스트의 모든 요소를 삭제하여 빈 리스트로 만들 수 있습니다.

▼ 3. 리스트 정렬하기

리스트를 정렬하는 방법에는 크게 두 가지가 있습니다.

```
numbers = [5, 2, 8, 1, 9, 3]

# 오름차순 정렬
numbers.sort()                # [1, 2, 3, 5, 8, 9]

# 내림차순 정렬
numbers.sort(reverse=True)    # [9, 8, 5, 3, 2, 1]

# 정렬된 새로운 리스트 생성
original = [5, 2, 8, 1, 9, 3]
new_list = sorted(original)   # original은 변경되지 않음
```

▼ sort

```
리스트.sort()
```

- sort()는 리스트를 직접 정렬하여 변경합니다.
- reverse=True 옵션을 사용하면 내림차순으로 정렬할 수 있습니다.

▼ sorted


```
sorted(리스트)
```

- sorted()는 원본 리스트는 그대로 두고, 정렬된 새로운 리스트를 반환합니다.
- 원본 데이터를 보존하고 싶을 때 유용합니다.

▼ 4. 리스트 확장하기

리스트를 확장하는 방법에는 extend()와 + 연산자를 사용하는 두 가지 방법이 있습니다.

```
list1 = [1, 2, 3]
list2 = [4, 5, 6]

# extend() 사용
list1.extend(list2)    # [1, 2, 3, 4, 5, 6]

# + 연산자 사용
new_list = list1 + list2    # [1, 2, 3, 4, 5, 6]
```

▼ extend()

```
리스트1.extend(리스트2)
```

- extend()는 리스트에 다른 리스트를 추가하여 하나의 리스트로 확장합니다.

▼ + 연산자

```
리스트1 + 리스트2
```

- 연산자를 사용하면 두 리스트를 연결한 새로운 리스트가 생성됩니다.

▼ 5. 값 존재 여부 확인하기

▼ in

`in` 을 사용하면 리스트에 특정 값이 존재하는지 확인할 수 있습니다.

```
fruits = ["사과", "바나나", "오렌지"]
if "바나나" in fruits:
```

```
print("바나나가 있습니다!")
```

- in을 사용하면 리스트에 특정 값이 있는지 빠르게 확인할 수 있습니다.

▼ count()

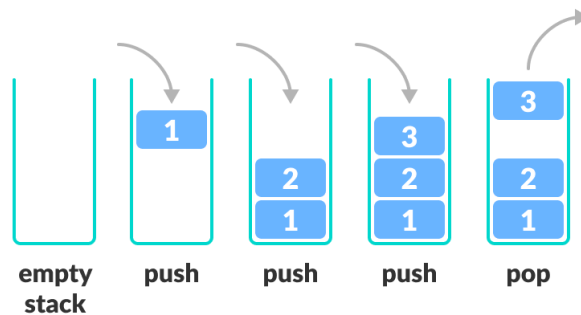
리스트에서 특정 값이 몇 번 등장하는지 확인하려면 count() 함수를 사용합니다.

```
numbers = [1, 2, 2, 3, 2, 4, 5, 2]
count_2 = numbers.count(2)    # 2가 나타난 횟수: 4
```

- count()는 리스트에서 특정 값이 몇 번 등장하는지 세어줍니다.
- 중복된 데이터를 찾거나 특정 값의 빈도를 확인할 때 유용합니다.

▼ 리스트로 만들 수 있는 자료구조

▼ 스택 (Stack)



스택은 프로그래밍에서 자주 사용되는 자료구조 중 하나로, 책을 쌓아두는 것처럼 데이터를 **차곡차곡 쌓아 올리는** 자료구조입니다.

▼ 스택의 특징

- 스택은 책을 쌓아두듯이 데이터를 순차적으로 쌓는 구조로, 새로운 데이터는 항상 맨 위에 추가됩니다.
- **가장 마지막에 들어온 데이터가 가장 먼저 나가는** LIFO (Last In First Out) 구조를 가집니다.
- 파이썬에서는 리스트를 활용하여 구현할 수 있습니다.

▼ 스택의 기본 스펙

▼ Push

- 스택의 가장 위에 새로운 데이터를 추가하는 연산입니다.
- 파이썬에서는 리스트의 `append()` 함수를 사용하여 구현할 수 있습니다.

▼ Pop

- 스택의 가장 위에 있는 데이터를 제거하고 반환하는 연산입니다.
- 파이썬에서는 리스트의 `pop()` 함수를 사용합니다.

```
stack = []
stack.append("A")           # push 하기
stack.append("B")           # push 하기
last_item = stack.pop()    # pop 하기
```

▼ 스택 활용 예시

- 데이터를 순차적으로 추가하고 가장 최근 것부터 꺼내서 사용하는 기능에 활용됩니다.
- ex) 웹 브라우저의 뒤로가기, 실행 취소(Undo) 기능

```
browser_stack = ["유튜브", "네이버", "구글"]
browser_stack.append("나무위키")           # 방문 히
스토리에 나무위키 추가
browser_stack.pop()                         # 뒤로가
기. 마지막에 방문한 페이지 삭제
```

▼ 큐 (Queue)



큐는 프로그래밍에서 자주 사용되는 자료구조 중 하나로, 줄을 서서 기다리는 것처럼 데이터를 **순서대로 처리**하는 자료구조입니다.

▼ 큐의 특징

- 큐는 마치 은행 창구처럼 먼저 온 데이터가 먼저 처리되는 구조입니다.
- **가장 먼저 들어온 데이터가 가장 먼저 나가는** FIFO(First In First Out) 구조를 가집니다.
- 파이썬에서는 리스트를 활용하여 구현할 수 있습니다.

▼ 큐의 기본 스펙

▼ Enqueue

- 큐의 맨 뒷 쪽에 새로운 데이터를 추가하는 연산입니다.
- 파이썬에서는 리스트의 `append()` 함수를 사용하여 구현할 수 있습니다.

▼ Dequeue

- 큐의 맨 앞 쪽에서 데이터를 제거하고 반환하는 연산입니다.
- 파이썬에서는 리스트의 `pop(0)` 함수를 사용합니다.

```
queue = []
queue.append("A")           # enqueue 하기
queue.append("B")           # enqueue 하기
first_item = queue.pop(0)   # dequeue 하기
```

▼ 큐 활용 예시

- 순서대로 처리해야 하는 작업이나 대기열 관리에 활용됩니다.
- ex) 프린터 인쇄 대기열, 은행 번호표 시스템

```
print_queue = ["문서1", "문서2", "문서3"]
print_queue.append("문서4")           # 새로운
문서 인쇄 요청 추가
next_print = print_queue.pop(0)       # 첫 번째
문서 인쇄 시작
```

▼ 문제

▼ 연습 문제: 숫자 뒤집기

리스트를 사용해서 입력받은 숫자의 순서를 거꾸로 만들어보세요.

```
numbers = [1, 2, 3, 4, 5]
reversed_numbers = []

# numbers를 뒤집은 결과가 reversed_numbers가 되도록,
# 여기에 코드를 작성하세요

print(reversed_numbers) # [5, 4, 3, 2, 1] 가 출력되어
야 함.
```

▼ 코드 예시 (참고만 하세요!)

```
numbers = [1, 2, 3, 4, 5]
reversed_numbers = []

# numbers를 뒤집은 결과가 reversed_numbers가 되도록,
# 여기에 코드를 작성하세요
reversed_numbers.append(numbers.pop())
reversed_numbers.append(numbers.pop())
reversed_numbers.append(numbers.pop())
reversed_numbers.append(numbers.pop())
reversed_numbers.append(numbers.pop())

print(reversed_numbers) # [5, 4, 3, 2, 1] 가 출력되
어야 함.
```

▼ 3. 반복문



반복문은 같은 일을 여러 번 실행해야 할 때 사용하는 구문입니다.
조건문만큼이나 매우 중요한 구문이며, 여러번 '노가다'하는 것을 획기적으로 줄여줄 수 있습니다.

▼ 반복문이란

- 반복문은 특정 코드를 여러 번 실행하게 해주는 구문입니다.
- 같은 코드를 여러 번 작성하는 대신, **반복문으로 간단하게 처리**할 수 있습니다.
- 파이썬에서는 **for문**과 **while문** 두 가지 반복문을 제공합니다.

▼ for 문

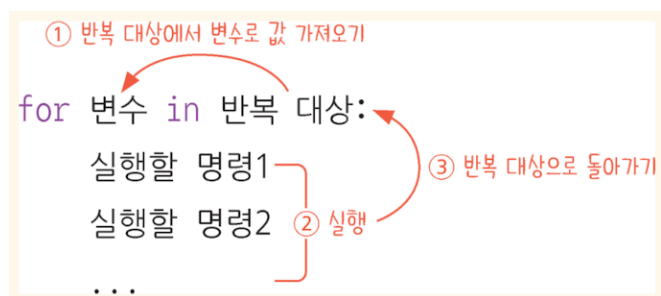


for 문을 사용하면, 여러 개의 값들을 **순회공연**하듯이 읽을 수 있습니다.



14,000,605 개의 미래를 for문으로 돌면서 타노스를 이길 방법을 찾는 닥터 스트레인지

▼ 기본 구조



```
for 변수 in 반복 대상:
    실행할 명령1
    실행할 명령2
    실행할 명령3
    ...
```

- **for**
반복을 시작한다는 명령어입니다.
- **변수**
반복할 때마다 각각의 값이 담기는 임시 변수입니다.
- **반복 대상**
리스트나 **range()** 등 여러 개의 값을 가진 데이터입니다.
반복할 대상이 더이상 없으면 for 전체는 끝이 납니다.

- **실행할 명령**
반복될 때마다 실행되는 명령입니다.
이 때 변수를 참조하여 사용할 수 있습니다.

▼ 사용 예시

```
possible_futures = [False, False, False, False, True]

for future in possible_futures:
    if future:
        print("타노스를 이기는 미래")
    else:
        print("타노스에게 지는 미래")
```

```
cities = ['서울', '대전', '부산', '인천', '광주']

for city in cities:
    print(f"{city}에서 임영웅 콘서트가 열립니다!")
```

- 위 코드를 실행하면 각 도시를 차례대로 순회하면서, 임영웅 콘서트를 개최합니다.
- for문은 리스트의 요소들을 차례대로 하나씩 읽어가며, 각 요소를 활용해서 특정한 작업을 수행할 수 있습니다.

```
# 운동 카운트
for count in range(3):
    print(f"{count + 1}번째 스쿼트!")
```

- **range(n)** : 0부터 n-1까지의 숫자를 생성해줍니다.
- 예를 들어 **range(5)** 는 0부터 4까지의 숫자를 만들어냅니다.

```
# 1부터 5까지 출력
for i in range(1, 6):
    print(i)
```


- `range(start, stop)` : start부터 stop-1까지의 숫자를 생성합니다.
- 예를 들어 `range(1, 6)` 은 1부터 5까지의 숫자를 만들어냅니다.
- `range(start, stop, step)` : start부터 stop-1까지 step만큼 증가하면서 숫자를 생성합니다.

```
# 2씩 증가 (0, 2, 4, 6, 8)
for i in range(0, 10, 2):
    print(i)
```

- 예를 들어 `range(0, 10, 2)` 는 0부터 8까지 2씩 증가하는 숫자를 만들어냅니다.

▼ for문을 멈추고 싶어요!



때로는 for 문을 중간에 멈추거나 특정 조건을 건너뛰어야 할 때가 있습니다.

이럴 때 사용하는 키워드가 바로 `break` 와 `continue` 입니다.

- **break**: 반복문을 완전히 종료하고 빠져나갑니다.
- **continue**: 현재 반복을 건너뛰고 다음 반복으로 넘어갑니다.

```
absent = [2, 5]                                # 결석한 학생 출석번호
no_book = [7]                                   # 책을 가져오지 않은 학생 출석번호

for student in range(1, 11):                    # 출석번호 1~10번
    if student in absent:
        # 결석한 학생이면
        continue
    # 다음 학생으로 넘어가기
    elif student in no_book:
        # 책을 가져오지 않으면 바로 수업 종료
        print(f"오늘 수업은 여기까지. {student}번 학생은 교무실로 따라와요 🙄")
        break
```

```
# 반복문 탈출
print(f"{student}번 학생, 책을 읽어 보세요.")
```

- **in**
반복할 대상을 지정합니다.

▼ for문으로 쉽게 리스트 만들기

[변수를 활용한 동작 for 변수 in 반복 대상]

- **리스트 컴프리헨션 (List Comprehension)**은 for문을 사용하여 쉽고 효율적으로 리스트를 생성하는 방법입니다.
- 복잡한 for문을 한 줄로 작성할 수 있어 코드가 더욱 깔끔해집니다.

```
avengers = ['iron man', 'thor', 'hulk', 'black widow']
upper_avengers = [hero.upper() for hero in avengers]

print(upper_avengers) # ['IRON MAN', 'THOR', 'HULK', 'BLACK WIDOW']
```

▼ for문 미니 실습

▼ 1) 이름 출력하기

```
# 과일 리스트
fruits = ["사과", "바나나", "딸기", "포도", "오렌지"]
```

좋아하는 과일 리스트와 for문을 사용해서 각 과일을 출력해보세요!

ex)

```
나는 사과를 좋아합니다!
나는 바나나를 좋아합니다!
나는 딸기를 좋아합니다!
나는 포도를 좋아합니다!
나는 오렌지를 좋아합니다!
```

▼ 샘플 코드

```
fruits = ["사과", "바나나", "딸기", "포도", "오렌지"]
```

```
for fruit in fruits:
    print(f"나는 {fruit}를 좋아합니다!")
```

▼ 2) 구구단 출력하기

for문을 사용하여 3단 구구단을 출력해보세요.

[출력 예시]

```
3 x 1 = 3
3 x 2 = 6
3 x 3 = 9
3 x 4 = 12
3 x 5 = 15
3 x 6 = 18
3 x 7 = 21
3 x 8 = 24
3 x 9 = 27
```

▼ 샘플 코드

```
# 3단만 찍기
for num in range(1, 10):
    print(f"3 X {num} = {num * 3}")

# 2-9단까지 전체 찍기
for dan in range(2, 10):
    for num in range(1, 10):
        print(f"{dan} X {num} = {num * dan}")
```

▼ 3) 숫자 합계 구하기

1부터 10까지의 숫자 합계를 구하는 간단한 for문을 작성해보세요.

```
result = 0

# 1부터 10까지의 숫자 합계를 구하는 for문을 작성해보세요

print(result)    # 출력 결과 55가 나와야 함
```

▼ 샘플 코드

```
result = 0

for num in range(1, 11):
    result += num

print(result)
```

▼ while 문



while 문은 특정 조건이 만족될 때까지 계속해서 코드를 반복할 수 있습니다.

▼ 기본 문법

while문의 기본 문법은 다음과 같습니다:

```
while 조건:
    실행할 명령1
    실행할 명령2
    ...
```

- 조건이 True인 동안 계속해서 명령들을 반복 실행합니다.
- 조건이 False가 되면 반복문이 종료됩니다.

▼ 예시

```
# 비밀번호 확인 예시
correct_password = "1234"
attempts = 0

while attempts < 3:
    password = input("비밀번호를 입력하세요: ")
    if password == correct_password:
        print("로그인 성공!")
```

```

        break
    else:
        print("비밀번호가 틀렸습니다.")
        attempts = attempts + 1

if attempts >= 3:
    print("비밀번호 최대 시도 횟수를 초과했습니다!")

```

▼ 루프를 영화를 떠올려봅시다



같은 일이 계속해서 반복되는 루프를 영화처럼,
while 문은 특정 조건이 되기 전까지 무한히 반복하는 구조입니다.

▼ 닥터 스트레인지



<https://www.youtube.com/watch?v=iNq-5nVTGbQ>

도르마무가 지구정복을 포기할 때까지,
닥터 스트레인지는 타임스톤으로 똑같은 순간을 계속 반복하면서 협박합니다.

```

dorumamu_convinced = False

while not dorumamu_convinced: #not은 뒤에 있는걸 반전

```

```
시킴, 즉 설득이 안됐을 때  
print("도르마무, 거래를 하러 왔다.")
```

▼ 엣지 오브 투모로우



주인공이 죽을 때마다 계속 똑같은 날이 반복됩니다.

▼ while문 사용 예시

- while 문은 반복 횟수가 특별히 정해지지 않은 경우에 특히 유용합니다.
- 메인 루프
우리가 쓰고 있는 대부분의 프로그램은 무한 반복 구조로 동작합니다.
프로그램이 종료될 때까지 사용자의 입력을 계속 기다리면서 처리하는 과정을 반복합니다.

▼ while문을 멈추고 싶어요!

- while문에서도 for문과 마찬가지로 `break` 와 `continue` 를 사용할 수 있습니다.
- `break` 는 반복문을 즉시 종료합니다.
- `continue` 는 현재 반복을 건너뛰고 다음 반복으로 넘어갑니다.

```
# break 예시
count = 0
while True:
    if count >= 5:
        break
    count += 1
    print(count)

# continue 예시
count = 0
while count < 5:
    count += 1
    if count == 3:
        continue
    else:
        print(count)
```

▼ 주의사항



while문은 조건이 만족되는 한 무한히 반복되기 때문에, 반드시 종료 조건을 잘 설정해야 합니다.

- 종료 조건이 잘 설정되지 않으면, 프로그램이 영원히 멈추지 않는 **무한 루프**에 빠질 수 있습니다.
- 무한루프에 빠지면 컴퓨터의 자원을 과도하게 사용하게 되어 프로그램이 멈추게 될 수도 있습니다.
(feat. 손난로앱)
- 그 무시무시한 도르마무도 while문 앞에서는 무릎을 꿇었습니다.

▼ 미니 실습

▼ 타이머 만들기

- n초 뒤에 알람을 울리는 타이머를 만들어 봅시다.
 - `input("시간을 초 단위로 입력해주세요")`
- 매 초 시간이 줄어들면서, 현재 남은 시간을 **시:분:초** 형태로 출력해야 합니다.

- 시간이 다 되면 `"🔔 시간이 다 되었습니다!"` 를 출력합니다.

▼ 힌트

예시

```
"0:0:3"
"0:0:2"
"0:0:1"
"🔔 시간이 다 되었습니다!"
```

▼ 1초 대기 하는 법

- `from time import *` 을 사용하면 `sleep()` 함수를 사용할 수 있습니다.
- `sleep(1)` 을 하면 1초 동안 프로그램을 멈춥니다.

▼ 초 단위를 분과 시로 변환하는 법

- 지난 시간에 배운 **나머지 연산**과 **몫 연산**을 활용해봅니다.
-

▼ 코드 예시 (참고만 하세요!)

```
from time import *

count = int(input("시간을 초 단위로 입력해주세요"))

# count가 0이 될 때까지
while count >= 0:
    # 시, 분, 초로 변환
    hours = count // 3600
    minutes = (count % 3600) // 60
    seconds = (count % 3600) % 60
    print(f"{hours}:{minutes}:{seconds}")

    # 카운트를 1 줄이고, 1초 기기
    count -= 1 # count = count - 1
    sleep(1)
```



```
print("🔔 시간이 다 되었습니다!")
```

▼ 실습 문제

두 개의 정렬된 리스트 합치기

이미 정렬되어 있는 두 개의 리스트를 하나의 정렬된 리스트로 합쳐봅시다!

▼ 조건

- 두 개의 정렬된 리스트가 주어집니다.
- 두 리스트를 합쳐서 하나의 정렬된 리스트를 만들어야 합니다.
- **결과 리스트도 정렬**되어야 합니다.
- sort 함수는 사용하면 안됩니다.

▼ 힌트

- `len (list)` → list의 길이

```
list1 = [1, 2, 3, 4]
len(list1) # 4가 나옵니다!
```

```
list2 = [1, 2, 3]
len(list2) # 3이 나옵니다!
```



▼ 예시

```
list1 = [1, 3, 5, 7]
list2 = [2, 4, 6, 8]

# 결과: [1, 2, 3, 4, 5, 6, 7, 8]

list1 = [1, 2, 4]
list2 = [1, 3, 4]

# 결과: [1, 1, 2, 3, 4, 4]
```

▼ 보너스 과제

1. 두 리스트의 길이가 다른 경우도 처리할 수 있어야 합니다
3. 중복된 숫자가 있는 경우도 처리할 수 있어야 합니다

▼ 코드 예시 (참고만 하세요!)

```
list1 = [1, 3, 5, 7]
list2 = [2, 4, 6, 8]

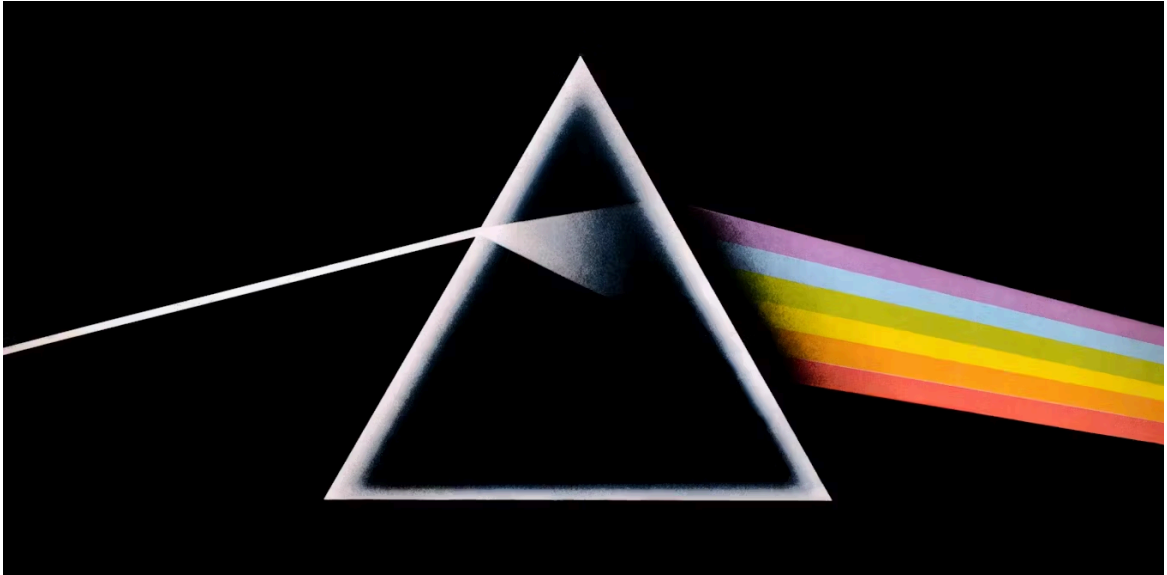
result = []

# list1과 list2가 비어있지 않는 한 계속해서 수행
while len(list1) > 0 and len(list2) > 0:
    # list1에 있는 현재 주자 vs list2에 있는 현재 주자
    # list1이 더 큰 경우
    if list1[0] < list2[0]:
        # result에 list1의 현재 주자를 빼서 넣는다
        result.append(list1.pop(0))
    # list2가 더 큰 경우
    else:
        result.append(list2.pop(0))
        # result에 list2의 현재 주자를 빼서 넣는다

# list1이나 list2 둘 중에 찌꺼기가 남아있으면 result에 추가
result.extend(list1)
result.extend(list2)
```

```
print(result)
```

▼ 4. 문자열 다루기



빛이 입자성과 파장성을 동시에 가진 것처럼, 문자열도 **리스트의 성질**을 가지고 있습니다.

▼ 문자열의 리스트 성질



사실 문자열도 일종의 리스트입니다!

문자들이 순서대로 나열되어 있어서 **마치 리스트처럼** 각 문자들을 다룰 수 있습니다

▼ 문자열 순회하기

문자열도 리스트처럼 for 문을 사용해서 각 문자를 순회할 수 있습니다.

```
text = "Python"
for char in text:
    print(char)  # P, y, t, h, o, n 각각 출력
```

▼ 문자열 인덱싱

문자열은 **인덱싱**과 **슬라이싱**이 가능합니다. 마치 리스트처럼 접근할 수 있습니다.

```

text = "ABCDEFGH"

text[0]          # 'A'

text[-1]         # 'H'

text[1:3]        # 'BC'

text[:3]         # 'ABC'

```

▼ 문자열의 위치와 횟수 검색

메소드	설명	예시	결과
<code>find()</code>	특정 문자열의 위치. 찾지 못하면 -1 반환	<code>"banana".find("na")</code>	2
<code>index()</code>	특정 문자열의 위치. 찾지 못하면 Error 발생	<code>"banana".index("na")</code>	2
<code>count()</code>	부분 문자열의 등장 횟수	<code>"banana".count("a")</code>	3

▼ 그 외 유용한 문자열 함수

함수	설명	예시	결과
<code>upper()</code> , <code>lower()</code>	대소문자 변환	<code>"Python".upper()</code>	"PYTHON"
<code>strip()</code>	양쪽 공백 제거	<code>" Python ".strip()</code>	"python"
<code>replace()</code>	문자열 치환	<code>"Python".replace("P", "J")</code>	"Jython"
<code>split()</code>	문자열 분리	<code>"1,2,3".split(",")</code>	['1', '2', '3']
<code>join()</code>	문자열 합치기	<code>"-".join(['A', 'B', 'C'])</code>	"A-B-C"

▼ 문자열 검색

▼ 값 존재 여부 확인하기

특정 문자나 부분 문자열이 포함되어 있는지 확인할 수 있습니다.

```

text = "Hello, World!"
print("o" in text)          # True

```

▼ 시작, 끝 문자열 확인

```
text = "Hello, World!"
print(text.startswith("Hello"))    # True
print(text.endswith("World"))      # True
```

▼ 그 외 유용한 문자열 함수

함수	설명	예시	결과
<code>upper()</code> , <code>lower()</code>	대소문자 변환	<code>"Python".upper()</code>	"PYTHON"
<code>strip()</code>	양쪽 공백 제거	<code>" Python ".strip()</code>	"python"
<code>replace()</code>	문자열 치환	<code>"Python".replace("P", "J")</code>	"Jython"
<code>split()</code>	문자열 분리	<code>"1,2,3".split(",")</code>	['1', '2', '3']
<code>join()</code>	문자열 합치기	<code>"-".join(['A', 'B', 'C'])</code>	"A-B-C"
<code>len()</code>	문자열 길이	<code>len("Hello")</code>	5

▼ 실습

스택으로 괄호 검사하기

스택을 활용하여 괄호의 짝이 맞는지 검사하는 프로그램을 만들어보세요.

```
brackets = "((()))"    # 올바른 괄호
# brackets = "(()"     # 올바르지 않은 괄호

stack = []

# 여기에 코드를 작성하세요
# 힌트: 여는 괄호는 스택에 추가, 닫는 괄호를 만나면 스택에서 제거

# brackets가 올바른 괄호일 경우, "올바른 괄호"라고 출력
# brackets가 올바르지 않은 괄호일 경우, "올바르지 않은 괄호"라고 출력
```

▼ 코드 예시 (참고만 하세요!)

```
brackets = ")"

stack = []

# for문을 활용해서 brackets을 처음부터 끝까지 돈다
for char in brackets:
    # 만난 문자가 만약에 여는 괄호라면
    if char == "(":
        # 스택에 추가(push)한다
        stack.append(char)
    # 만난 문자가 만약에 닫는 괄호라면
    elif char == ")":
        # 스택이 비어있지 않은 경우
        if stack:
            # 스택에서 제거(pop)한다
            stack.pop()
        # 스택이 비어 있는 경우
        else:
            # 정상적이지 않은 괄호!
            stack.append(char)
            break

# for문이 다 끝난 뒤, stack에 찌꺼기가 남아 있으면
if stack:
    # 정상적이지 않은 괄호!
    print("정상적이지 않은 괄호!")
# for문이 다 끝난 뒤, stack이 비어있으면
else:
    # 정상적인 괄호!
    print("정상적인 괄호!")
```